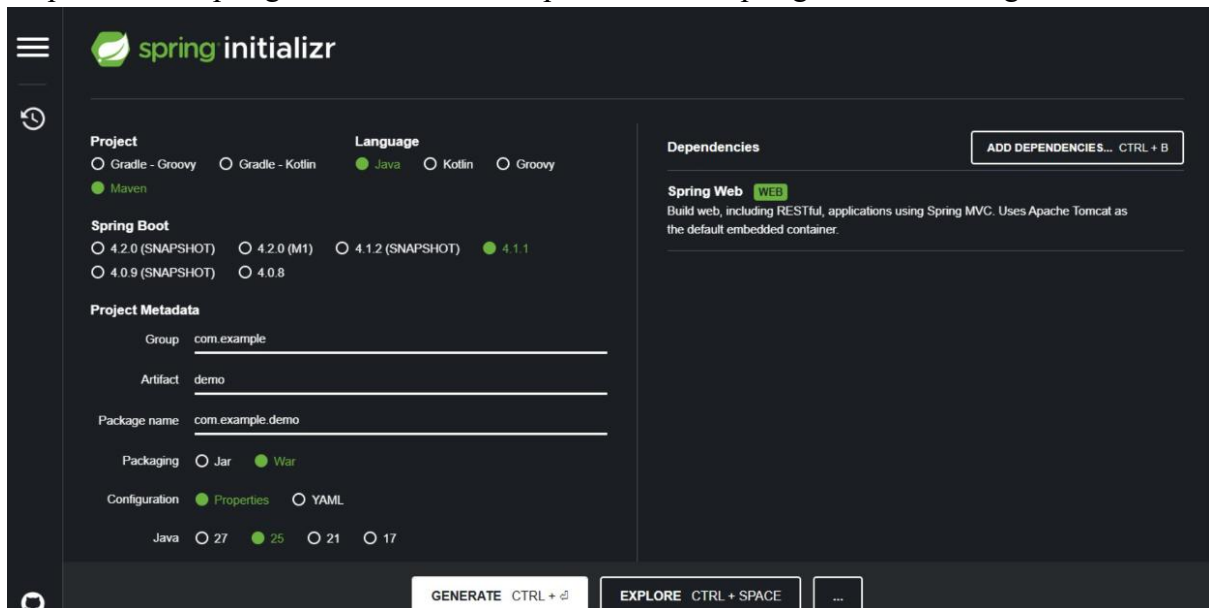


RestAPI's using springboot and postman

Step 1 : Go to spring initializr and add dependencies of spring web , click on generate

The screenshot shows the Spring Initializr web application interface. It has a dark theme. On the left, there's a sidebar with a hamburger menu icon and a circular arrow icon. The main area is divided into sections: 'Project' with radio buttons for 'Gradle - Groovy', 'Gradle - Kotlin', 'Java' (selected), and 'Groovy'; 'Maven' (selected); 'Spring Boot' with radio buttons for '4.2.0 (SNAPSHOT)', '4.2.0 (M1)', '4.1.2 (SNAPSHOT)', '4.1.1' (selected), and '4.0.9 (SNAPSHOT)'; '4.0.8'; 'Project Metadata' with input fields for 'Group' (com.example), 'Artifact' (demo), and 'Package name' (com.example.demo); 'Packaging' with radio buttons for 'Jar' and 'War' (selected); 'Configuration' with radio buttons for 'Properties' (selected) and 'YAML'; and 'Java' with radio buttons for '27', '25' (selected), '21', and '17'. On the right, there's a 'Dependencies' section with a button 'ADD DEPENDENCIES... CTRL + B'. Below it, 'Spring Web' is selected with a 'WEB' tag, and a description says 'Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.' At the bottom, there are buttons 'GENERATE CTRL + G', 'EXPLORE CTRL + SPACE', and a three-dot menu.

Step 2 : open IntelliJ idea and open the dependencies file

Step3 : create the class as cabbookingApplication.java and develop a code as  
package com.example.CabBooking;

```
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
import org.springframework.web.bind.annotation.*;
```

```
import java.util.ArrayList;  
import java.util.List;
```

```
@SpringBootApplication  
public class CabBookingApplication {  
  
    public static void main(String[] args) {  
        SpringApplication.run(CabBookingApplication.class, args);  
    }  
}
```

```
}
```

```
@RestController
```

```
@RequestMapping("/api/cabs")
```

```
class CabController {
```

```
    private final List<String> cl = new ArrayList<>(List.of("Cab Booking 1", "Cab Booking 2"));
```

```
    @GetMapping
```

```
    public List<String> getcl() {
```

```
        return cl;
```

```
}
```

```
    @PostMapping
```

```
    public String addCab(@RequestBody String newCab) {
```

```
        cl.add(newCab);
```

```
        return "cab added successfully " + newCab;
```

```
}
```

```
    @PutMapping("/{id}")
```

```
    public String updateCab(@PathVariable int id, @RequestBody String updatedCab) {
```

```
        if (id >= 0 && id < cl.size()) {
```

```
            cl.set(id, updatedCab);
```

```
            return "cab at index " + id + " updated to " + updatedCab;
```

```
        }
```

```
        return "task not found";
```

```
}
```

```

@DeleteMapping("/{id}")
public String deleteCab(@PathVariable int id) {
    if (id >= 0 && id < cl.size()) {
        String removed = cl.remove(id);
        return "deleted cab " + removed;
    }
    return "task not found";
}
}

```

Step 4 : observe the code in servletapplication.java and develop as follows

package com.example.CabBooking;

```

import org.springframework.boot.builder.SpringApplicationBuilder;
import org.springframework.boot.web.servlet.support.SpringBootServletInitializer;

```

```

public class ServletInitializer extends SpringBootServletInitializer {

```

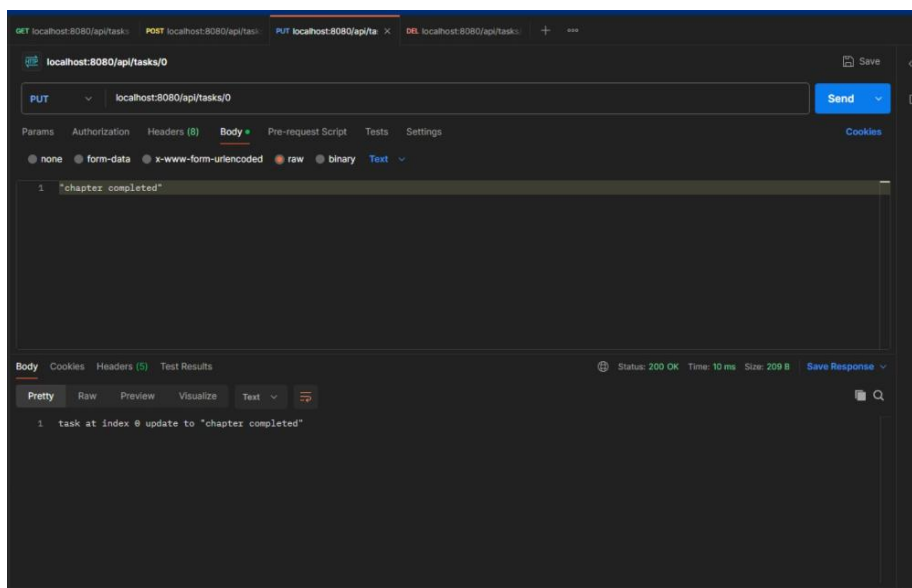
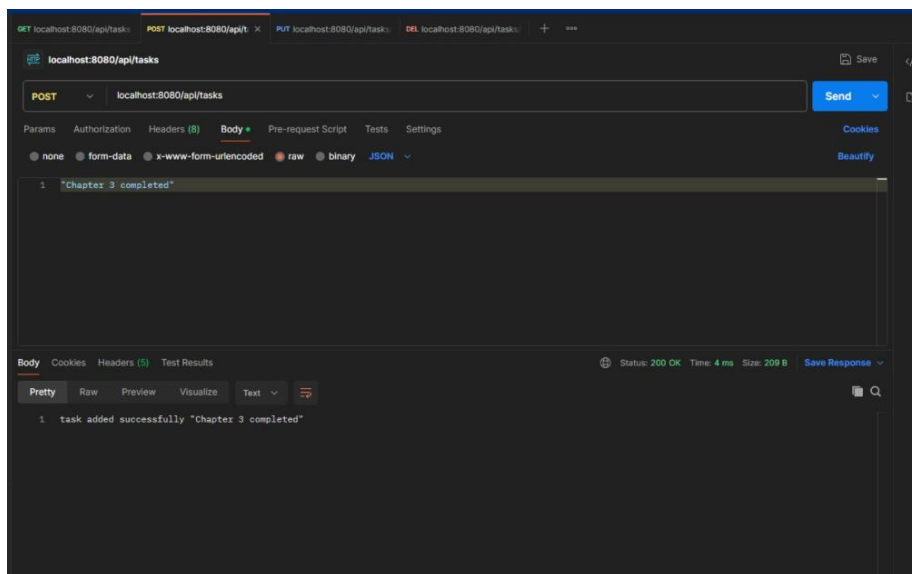
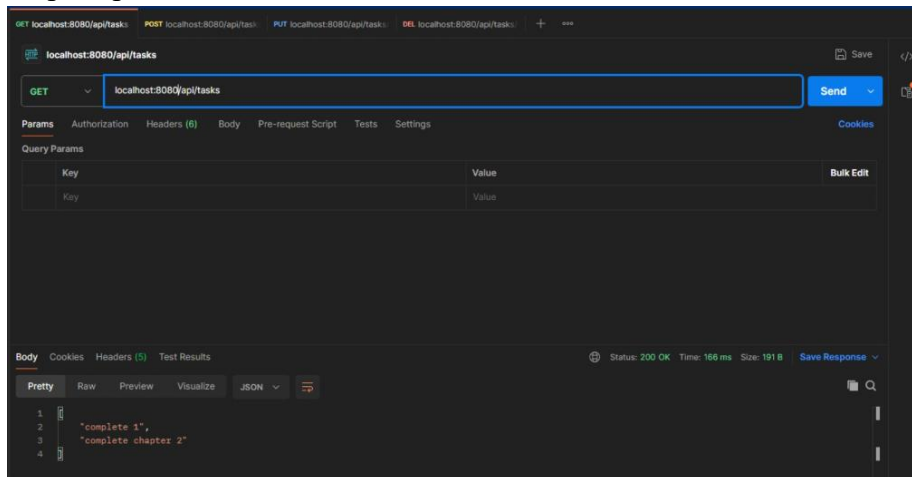
```

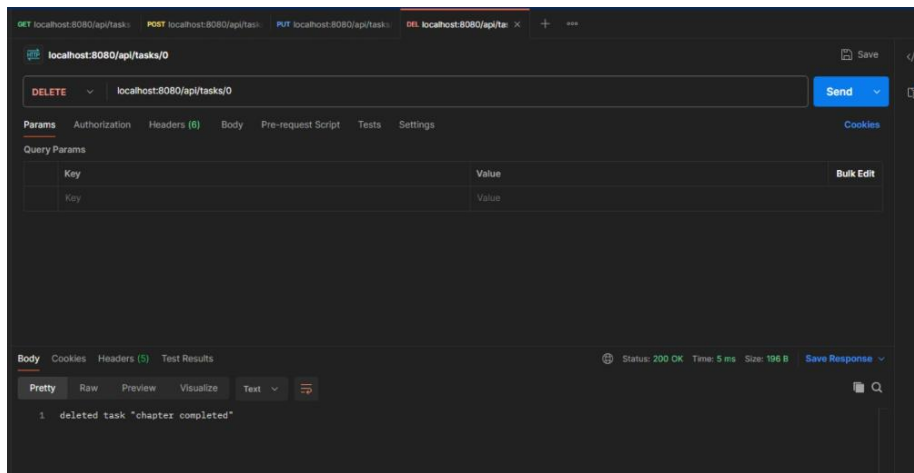
    @Override
    protected SpringApplicationBuilder configure(SpringApplicationBuilder application) {
        return application.sources(CabBookingApplication.class);
    }
}

```

Step 5 : open post man and paste the url using portno shown in springboot (localhost:8080)

## Step 6 : perform all the API's methods GET,POST,PUT,DELETE





Observe the body to change for Json while performing POST & PUT methods